

# Ferry Capacity Utilization and Operational Efficiency Analytics: A Data-Driven Framework for Passenger Demand Monitoring and Forecasting on the Toronto Island Ferry System

Nidhi

*Independent Researcher / Developer*

GitHub: <https://github.com/22bcs16147>

Project Repository: <https://github.com/22bcs16147/Ferry-Capacity-Utilization-Analytics>

Live Dashboard: <https://ferry-capacity-utilization-analytics-nidhi.streamlit.app/>

---

## Abstract

Urban ferry systems operate under fluctuating passenger demand that is difficult to monitor without a structured analytical pipeline. This paper presents the design and implementation of the Ferry Capacity Utilization and Operational Efficiency Analytics System, an end-to-end analytics solution built on eleven years of interval-level ticketing records from the Toronto Island Ferry service. The system ingests 261,538 raw ticket-activity records spanning May 2015 to December 2025 and applies a structured pipeline consisting of data understanding, temporal feature engineering, exploratory data analysis, key performance indicator (KPI) computation, interactive dashboard delivery, and short-horizon demand forecasting. Eight operational KPIs — Total Sales, Total Redemption, Total Activity Load, Capacity Utilization, Idle Capacity, Average Operational Load Index, Peak Activity, and Redemption Pressure Ratio — are derived directly from sales and redemption counts and exposed through an interactive Streamlit dashboard with six filter dimensions (date, year, month, season, time band, weekday/weekend) and twelve chart-level views, including yearly, monthly, seasonal, hourly, and heatmap representations. A Facebook Prophet model with yearly and weekly seasonality components is trained on aggregated daily activity to produce a rolling 30-day forecast, which is served alongside historical data inside the same dashboard. Analysis of the processed dataset shows that capacity utilization across the observed period sits near 98.6 percent, that idle low-activity intervals account for close to 45 percent of all 15-minute intervals, and that demand is concentrated in the midday hours and summer months, with August alone accounting for the largest single-month share of activity. The resulting system demonstrates how a lightweight, open-source analytics stack — Pandas, NumPy, Plotly, Streamlit, and Prophet — can be assembled into a reproducible operational intelligence tool without requiring proprietary business intelligence software, and the paper concludes with operational recommendations and directions for extending the forecasting and dashboard components.

**Keywords:** *ferry operations analytics, capacity utilization, time-series forecasting, Facebook Prophet, Streamlit dashboard, exploratory data analysis, transportation KPI, feature engineering*

## 1. Introduction

Public transit operators, including marine transit services, increasingly rely on ticketing systems that record activity at fine temporal granularity. The Toronto Island Ferry service records sales and redemption counts at 15-minute intervals, producing a dense operational log that, if left unexamined, yields little actionable insight beyond raw counts. The Ferry Capacity Utilization and Operational Efficiency Analytics System was developed to convert this raw interval-level log into a structured decision-support tool for operational planning.

The motivation for the project is twofold. First, ferry operators need a compact set of performance indicators that summarize demand and capacity behavior without requiring manual spreadsheet analysis of hundreds of thousands of rows. Second, operators benefit from a forward-looking view of demand, since staffing and scheduling decisions must be made ahead of the observed activity. This project addresses both needs by combining descriptive analytics (KPIs, charts, an interactive dashboard) with a predictive component (a Prophet-based 30-day forecast), all delivered through a single Streamlit application that can be filtered interactively by date, year, month, season, time band, and weekday/weekend status.

This paper documents the system as implemented, describing the dataset, the processing pipeline, the KPIs and their formulas, the dashboard architecture, and the forecasting methodology, and reports the results obtained when the pipeline is run against the full historical dataset.

## **2. Literature Review**

Transit analytics research has generally progressed along two parallel tracks: descriptive dashboards for operational monitoring, and time-series models for demand prediction.

On the descriptive side, interactive dashboard frameworks such as Streamlit and Plotly have lowered the barrier for building filterable, chart-driven monitoring tools without a dedicated business-intelligence license, allowing analysts to expose KPI cards, trend charts, and drill-down tables directly from a Pandas DataFrame. This aligns with the approach followed in the present project, where the entire dashboard layer is implemented in a single Streamlit script drawing on a pre-processed CSV file.

On the predictive side, additive time-series decomposition models have become a common choice for demand forecasting in operational settings where the analyst needs an interpretable model rather than a black-box neural forecaster. Facebook's Prophet library, used in this project, decomposes a series into trend, yearly seasonality, and weekly seasonality components, which is well suited to ferry ticket activity that plausibly exhibits both a yearly cycle (peak summer tourism season) and a weekly cycle (weekend versus weekday ridership).

Feature-engineering practices common in transportation analytics — deriving hour-of-day, day-of-week, weekend/weekday flags, and season labels from a timestamp — are also reflected directly in this project's `feature_engineering.py` module, which constructs Year, Quarter, Month, Day, Hour, Weekend, Season, and Time Band fields from the raw Timestamp column before any KPI or chart computation takes place.

The present work does not claim methodological novelty in the choice of Prophet or Streamlit; its contribution is the assembly of a complete, reproducible pipeline — from raw CSV to production dashboard with forecasting — tailored specifically to the Toronto Island Ferry ticketing dataset.

## **3. Problem Statement**

The raw Toronto Island Ferry Ticket dataset consists of 261,538 rows, each identified by an `_id`, a Timestamp, a Redemption Count, and a Sales Count. In this raw form, the dataset offers no derived operational context: there is no year, season, or time-of-day label, no measure of total passenger activity per interval, and no summarized indicator of how efficiently ferry capacity is being used. Manually aggregating over a quarter-million rows to answer operational questions such as “which hours are busiest,” “how much capacity sits idle,” or “what will demand look like next month” is impractical without a dedicated pipeline. The problem this project addresses is the absence of a structured, repeatable, and interactively explorable analytics layer over this ticketing dataset —

one that produces standardized KPIs, exposes them through filters, and extends historical observation with a short-term forecast.

## 4. Objectives

The system was built with the following objectives, each of which is reflected directly in a corresponding project module:

1. Understand the structure, data types, date range, and integrity (absence of negative values) of the raw ticketing dataset (data\_understanding.py).
2. Engineer temporal and business-relevant features — Year, Quarter, Month, Day, Hour, Weekend/Weekday, Season, Time Band, Total Activity Load, Redemption Pressure Ratio, Operational Load Index, and Idle Capacity Indicator — from the raw timestamp and count fields (feature\_engineering.py).
3. Explore the processed dataset visually to identify yearly, monthly, weekend/weekday, and time-band activity patterns (exploratory\_analysis.py).
4. Compute a standardized set of KPIs describing capacity utilization, idle capacity, congestion pressure, and operational variability (kpi\_analysis.py).
5. Deliver the KPIs, charts, and insights through an interactive, filterable Streamlit dashboard (dashboard/app.py).
6. Forecast passenger demand 30 days ahead using Facebook Prophet, trained on daily-aggregated activity, and surface that forecast inside the same dashboard (forecasting.py).
7. Translate the resulting KPIs and trends into operational recommendations for ferry scheduling, staffing, and capacity planning.

## 5. Dataset Description

The dataset used is the Toronto Island Ferry Ticket Activity dataset, provided as a single CSV file (Toronto Island Ferry Tickets.csv) with four raw columns: `_id`, `Timestamp`, `Redemption Count`, and `Sales Count`. Verification performed as part of the data-understanding stage confirmed that the dataset contains no negative sales or redemption values, and that the timestamp column spans from May 1, 2015 to December 21, 2025 — eleven distinct calendar years of interval-level activity.

**Table 1. Dataset Overview**

| Attribute                | Value                                                                                                |
|--------------------------|------------------------------------------------------------------------------------------------------|
| Source file              | Toronto Island Ferry Tickets.csv                                                                     |
| Raw columns              | <code>_id</code> , <code>Timestamp</code> , <code>Redemption Count</code> , <code>Sales Count</code> |
| Total records            | 261,538                                                                                              |
| Recording interval       | 15 minutes                                                                                           |
| Date range               | 2015-05-01 to 2025-12-21                                                                             |
| Years covered            | 11                                                                                                   |
| Total ticket sales       | 12,972,051                                                                                           |
| Total ticket redemptions | 12,785,293                                                                                           |
| Negative-value records   | 0 (verified in data_understanding.py)                                                                |

| Attribute                 | Value                      |
|---------------------------|----------------------------|
| Processed dataset columns | 19 (raw 4 + 15 engineered) |

## 6. Methodology

The methodology follows a linear pipeline: raw data is understood and validated, engineered into analytical features, explored visually, summarized into KPIs, delivered through a dashboard, and finally extended with a forecast. Each stage corresponds to one script in the `src/` directory or the `dashboard/` folder.

### 6.1 Data Collection

The raw dataset is read directly from `data/raw/Toronto Island Ferry Tickets.csv` using Pandas. The Timestamp field, stored as a string, is converted to a proper datetime object at the start of every downstream script (`data_understanding.py`, `feature_engineering.py`, `forecasting.py`), which is a prerequisite for all subsequent date-based grouping and filtering.

### 6.2 Data Cleaning

The `data_understanding.py` script performs the data-quality checks that precede feature engineering. It reports the dataset shape and column data types, computes the minimum and maximum timestamps to confirm the observation window, and explicitly counts negative values in the Sales Count and Redemption Count fields, confirming that both counts are non-negative throughout the dataset. Descriptive statistics (`describe()`), along with column-wise minimum and maximum values, are also printed to surface any structural anomalies before the data is passed to the feature-engineering stage.

### 6.3 Feature Engineering

The `feature_engineering.py` module derives fifteen additional columns from the two raw count fields and the timestamp, which are then persisted to `data/processed/ferry_processed.csv` for use by every downstream stage. The derived fields fall into three groups: calendar/time features, categorical bucket features, and business-metric features.

**Table 2. Feature Engineering Summary**

| Feature                          | Derivation                                                             | Purpose                                         |
|----------------------------------|------------------------------------------------------------------------|-------------------------------------------------|
| Year, Quarter, Month, Month Name | Extracted from Timestamp                                               | Enables yearly and monthly aggregation          |
| Day, Day Name                    | Extracted from Timestamp                                               | Enables day-level and weekday-name grouping     |
| Hour, Minute                     | Extracted from Timestamp                                               | Enables hourly and sub-hourly activity analysis |
| Weekend                          | “Weekend” if Day Name is Saturday/Sunday, else “Weekday”               | Distinguishes weekday from weekend demand       |
| Season                           | Winter (Dec–Feb), Spring (Mar–May), Summer (Jun–Aug), Autumn (Sep–Nov) | Groups activity by calendar season              |
| Time Band                        | Morning (5–11h), Afternoon (12–16h), Evening (17–20h), Night (21–4h)   | Groups activity into four operational bands     |

| Feature                   | Derivation                                        | Purpose                                                                      |
|---------------------------|---------------------------------------------------|------------------------------------------------------------------------------|
| Total Activity Load       | Sales Count + Redemption Count                    | Composite measure of interval-level passenger activity                       |
| Redemption Pressure Ratio | Redemption Count / (Sales Count + 1)              | Indicates how redemption volume compares to sales volume per interval        |
| Operational Load Index    | Total Activity Load / max(Total Activity Load)    | Normalizes activity to a 0–1 scale relative to the busiest interval observed |
| Idle Capacity Indicator   | “Idle” if Total Activity Load < 20, else “Active” | Flags low-activity (under-utilized) 15-minute intervals                      |

The idle threshold of 20 combined passenger-activity units per 15-minute interval is a fixed rule encoded directly in the feature-engineering script and is used consistently by both the KPI-analysis stage and the dashboard's idle-capacity metric.

## 6.4 Exploratory Data Analysis

The `exploratory_analysis.py` script loads the processed dataset and, after configuring a consistent Matplotlib/Seaborn visual style, prints a dataset overview (record count, year count, total sales, total redemption) and generates four static charts saved to `assets/charts/`: a yearly activity line chart, a monthly activity bar chart (ordered chronologically rather than alphabetically), a weekend-versus-weekday bar chart, and a time-band bar chart. These four exploratory charts establish the baseline patterns — yearly trend, seasonal concentration, weekday/weekend split, and time-of-day concentration — that are later reproduced interactively, with additional dimensions, in the Streamlit dashboard.

## 6.5 KPI Development

The `kpi_analysis.py` script computes a console-based KPI summary from the processed dataset, forming the analytical basis for the dashboard's metric cards. Beyond the eight headline dashboard KPIs, this script also computes three additional diagnostic measures — Congestion Pressure Index, Operational Variability Score, and the raw Congestion Threshold — that characterize how concentrated and how variable the observed activity is.

**Table 3. Key Performance Indicators**

| KPI                  | Formula                                                     | Rationale                                                                                              |
|----------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Total Sales          | $\Sigma$ Sales Count                                        | Baseline demand-side ticketing volume                                                                  |
| Total Redemption     | $\Sigma$ Redemption Count                                   | Baseline usage-side ticketing volume                                                                   |
| Total Activity Load  | $\Sigma$ (Sales Count + Redemption Count)                   | Combined measure of ferry-related passenger activity                                                   |
| Capacity Utilization | $(\text{Total Redemption} / \text{Total Sales}) \times 100$ | Indicates how closely redemptions track ticket sales, a proxy for how fully purchased capacity is used |
| Idle Capacity        | % of intervals flagged “Idle” (Total Activity Load < 20)    | Quantifies the share of time the system operates near-empty                                            |

| KPI                            | Formula                                                   | Rationale                                                                           |
|--------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------------------------|
| Average Operational Load Index | Mean of (Total Activity Load / max Total Activity Load)   | Normalized sense of typical load relative to peak observed load                     |
| Peak Activity                  | max(Total Activity Load)                                  | Identifies the single busiest recorded interval, useful for capacity-sizing         |
| Redemption Pressure Ratio      | Mean of Redemption Count / (Sales Count + 1) per interval | Flags intervals where redemption volume is disproportionate to same-interval sales  |
| Congestion Pressure Index      | % of intervals at/above the 90th percentile of activity   | Measures how much of total operating time is spent under high-congestion conditions |
| Operational Variability Score  | Standard deviation of Total Activity Load                 | Captures how unevenly activity is distributed across intervals                      |

## 6.6 Dashboard Development

The dashboard is implemented as a single Streamlit script, `dashboard/app.py`, using `st.cache_data` to load the processed CSV once per session. The sidebar exposes six independent filters — a date-range picker, and multiselect controls for year, month, season, time band, and weekday/weekend — that are applied sequentially to a working copy of the DataFrame (`filtered_df`) before any KPI or chart is computed, so that every downstream view reflects the user's current filter selection.

The dashboard body is organized into the following sections, in the order they appear in the script: eight KPI metric cards (arranged as two rows of four `st.metric` cards); an Executive Summary panel reporting the analysis period, years covered, busiest year, peak hour, busiest season, and average daily activity; six primary Plotly charts (Yearly Activity Trend, Monthly Activity, Seasonal Activity as a pie chart, Time Band Analysis, Weekend vs. Weekday, and Hourly Activity); a Top 10 Busiest Hours and Top 10 Least Busy Hours pair of ranked bar charts; a Top 10 Busiest Days and Top 10 Least Busy Days pair; a Monthly Performance Ranking table with a quartile-based “Excellent/Good/Average/Low” performance label; an Operational Efficiency Score panel; an Hourly Congestion Analysis chart that labels each hour “High,” “Moderate,” or “Low” congestion based on the 75th and 25th percentiles of hourly activity; a Peak Demand Summary; a Monthly Growth Analysis chart showing month-over-month percentage change per year; a Yearly Performance Ranking bar chart; a Month-versus-Hour activity heatmap; a Business Insights and Operational Recommendations panel; a CSV download button for the filtered dataset; and, finally, the 30-day Prophet forecast chart with its own CSV download button.

The Operational Efficiency Score is computed as a weighted combination of three normalized inputs already present in the KPI layer:

$$\text{Efficiency Score} = 0.5 \times \text{Capacity Utilization} + 0.3 \times (100 - \text{Idle Capacity}) + 0.2 \times (\text{Average Redemption Pressure Ratio} \times 100), \text{ capped at } 100.$$

The resulting score is mapped to a three-tier status label (“Excellent” at 85+, “Good” at 70–84, “Needs Improvement” below 70), giving operators a single composite figure alongside the eight underlying KPIs.

## 6.7 Forecasting using Facebook Prophet

The forecasting.py script re-loads the processed dataset, aggregates Total Activity Load by calendar date to produce a daily time series (ds, y), and fits a Prophet model configured with yearly seasonality enabled, weekly seasonality enabled, and daily seasonality disabled. This configuration was chosen because the aggregation step collapses each day to a single value, removing any genuine intra-day seasonal signal, while the yearly and weekly components correspond to the two cyclical patterns already visible in the exploratory analysis — the summer-tourism-driven annual cycle and the weekday/weekend cycle. The fitted model is extended 30 days beyond the last observed date using make\_future\_dataframe, and the resulting predictions (yhat, with yhat\_lower and yhat\_upper uncertainty bounds) are written to assets/forecast/forecast\_results.csv. A Plotly line chart overlaying the forecast on the historical daily series is additionally saved as a standalone forecast.html file, and the dashboard separately renders the final 30 forecasted rows as both a chart and a downloadable table.

## 7. System Architecture

The system follows a five-layer batch-then-serve architecture, in which two offline scripts (feature engineering and forecasting) prepare artifacts that a live Streamlit process subsequently reads and serves interactively.

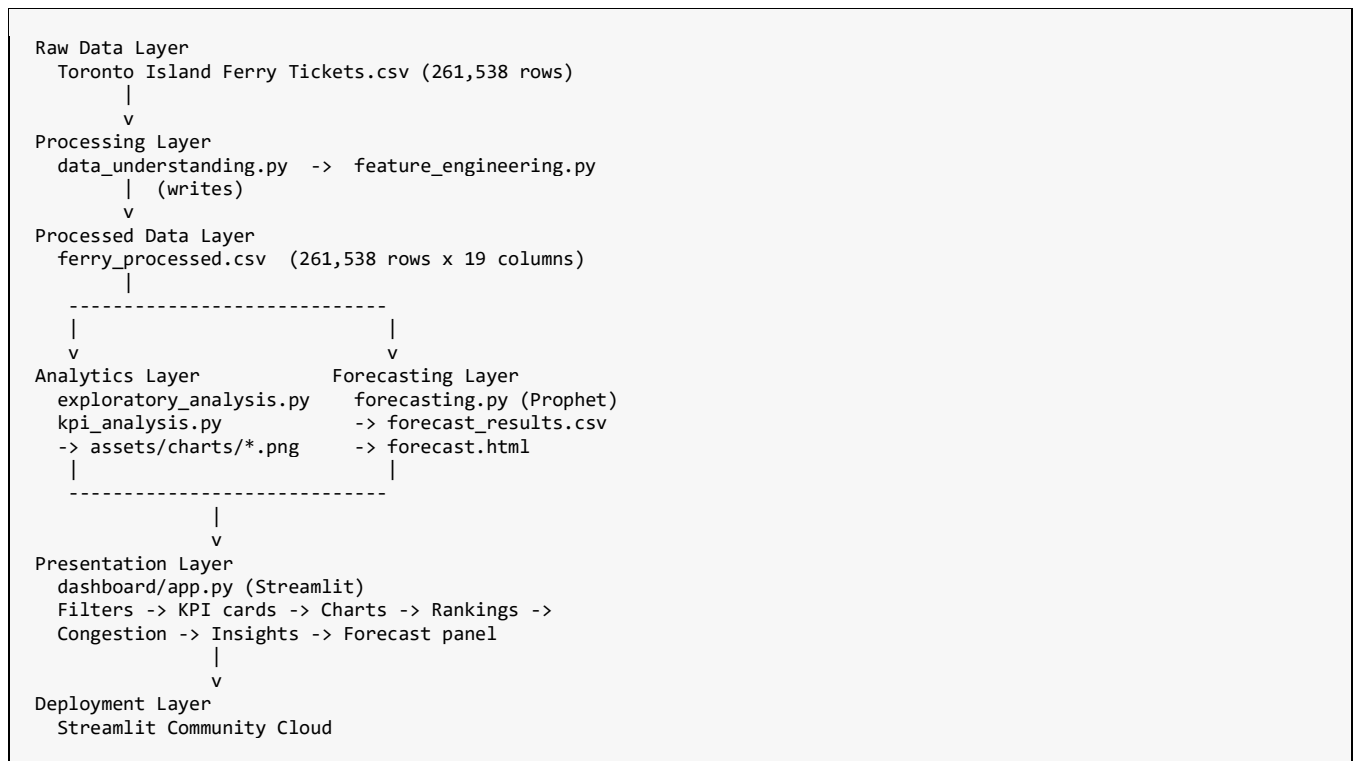
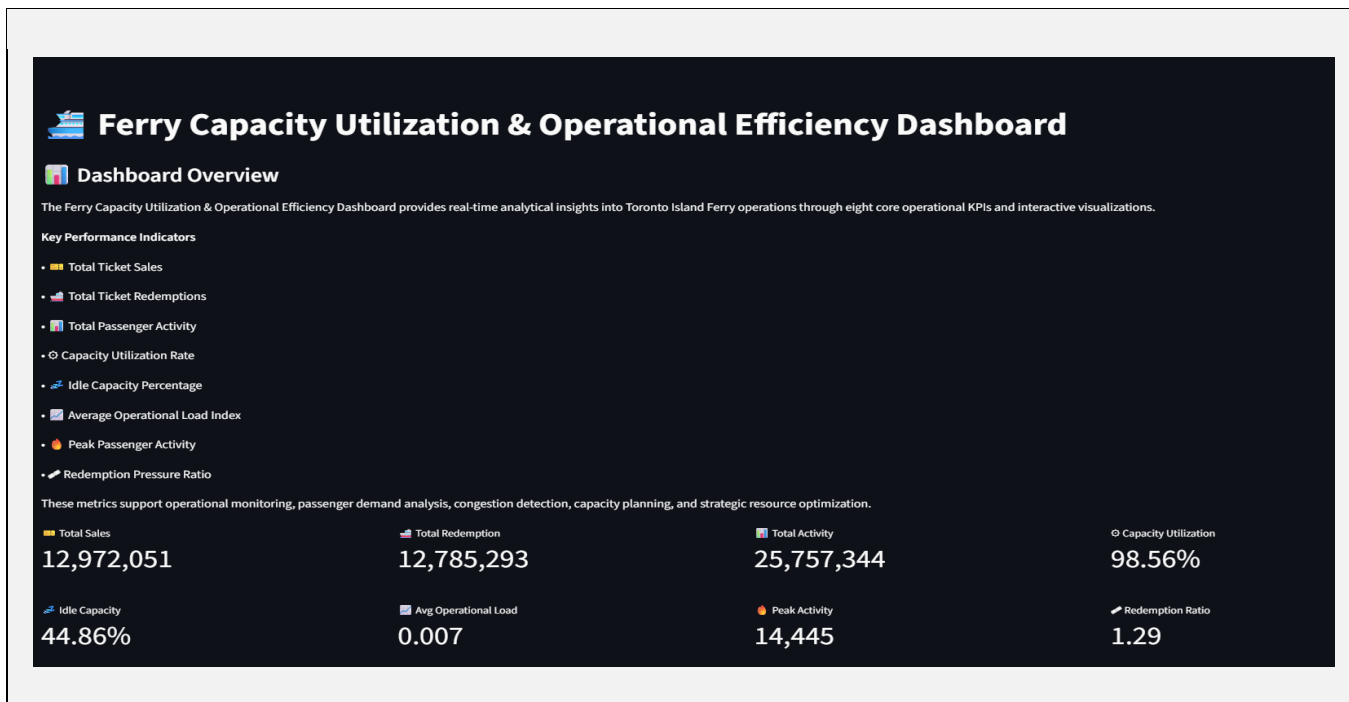


Figure 1. Dashboard Home Page



Each layer reads only from the output of the layer above it: the analytics and forecasting layers both depend solely on `ferry_processed.csv`, and the Streamlit presentation layer depends on the processed CSV plus the pre-computed `forecast_results.csv`, rather than recomputing the Prophet model on every dashboard session — a design choice that keeps interactive filtering responsive.

## 8. Results

Running the full pipeline against the historical dataset yields the KPI values summarized below (computed on the complete, unfiltered processed dataset).

**Table 4. KPI Results on the Full Dataset**

| KPI                                                  | Value      |
|------------------------------------------------------|------------|
| Total Ticket Sales                                   | 12,972,051 |
| Total Ticket Redemptions                             | 12,785,293 |
| Total Activity Load                                  | 25,757,344 |
| Average Activity per Interval                        | 98.48      |
| Capacity Utilization Ratio                           | 98.56%     |
| Idle Capacity Percentage                             | 44.86%     |
| Congestion Pressure Index ( $\geq 90$ th percentile) | 10.03%     |
| Operational Variability Score (std. dev.)            | 200.19     |
| Peak Activity (single interval)                      | 14,445     |
| Average Operational Load Index                       | 0.0068     |
| Average Redemption Pressure Ratio                    | 1.29       |

Two results stand out. First, the Capacity Utilization Ratio of 98.56 percent indicates that, in aggregate, redemption volume tracks sales volume almost one-for-one across the eleven-year period — consistent with the

ferry operating as a same-day or short-window ticketing service rather than one with large backlogs of unredeemed tickets. Second, the Idle Capacity Percentage of 44.86 percent shows that despite the high aggregate utilization ratio, nearly half of all 15-minute intervals fall below the 20-unit activity threshold, meaning idle time is concentrated rather than evenly distributed — a pattern confirmed by the hourly and time-band breakdowns below.

**Table 5. Activity Distribution by Time Band**

| Time Band          | Total Activity Load | Share |
|--------------------|---------------------|-------|
| Afternoon (12–16h) | 13,098,123          | 50.9% |
| Morning (5–11h)    | 7,566,895           | 29.4% |
| Evening (17–20h)   | 4,126,536           | 16.0% |
| Night (21–4h)      | 965,790             | 3.7%  |

**Table 6. Seasonal Activity Distribution**

| Season | Total Activity Load | Share |
|--------|---------------------|-------|
| Summer | 16,341,976          | 63.5% |
| Autumn | 5,105,037           | 19.8% |
| Spring | 3,358,672           | 13.0% |
| Winter | 951,659             | 3.7%  |

The hour-level breakdown shows the single busiest hour of the day to be 12:00 (noon), with 2,932,845 cumulative activity units, followed closely by 13:00, 11:00, and 14:00 — all within the afternoon band. The least active hours are 04:00 and 05:00, each accounting for well under 10,000 cumulative activity units across the entire eleven-year history, confirming the Night band's marginal 3.7 percent share. At the month level, August is the single busiest month (6,821,385 activity units), followed by July, June, and September, while January is the least active month (282,075 units) — a pattern that lines up directly with the Season-level Summer concentration above.

While the raw sum of activity is higher on weekdays (15,002,222) than on weekends (10,755,122) simply because there are more weekdays than weekend days in any date range, normalizing by the number of calendar days in each category reverses this picture: average activity per weekend day (9,706.8) is roughly 1.8 times higher than average activity per weekday (5,408.2), confirming that per-day demand is weekend-skewed even though weekday totals are larger in absolute terms.

Figure 2. Yearly Activity Trend



Figure 3. Monthly Activity

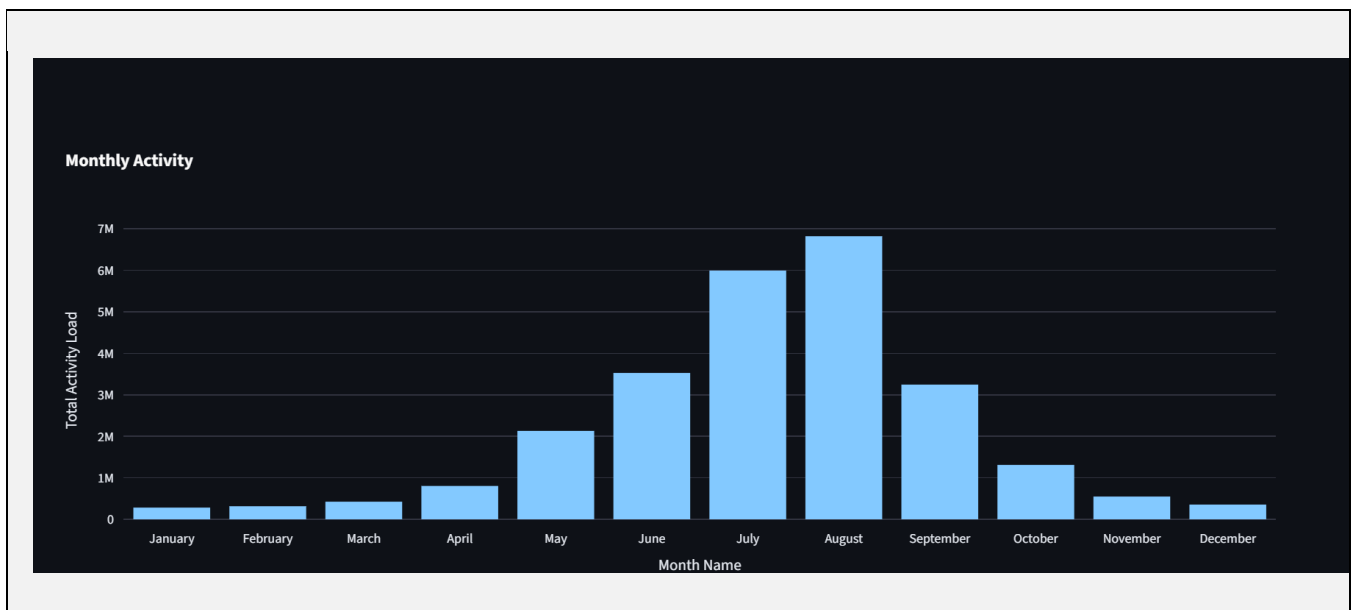


Figure 4. Weekend vs. Weekday Activity

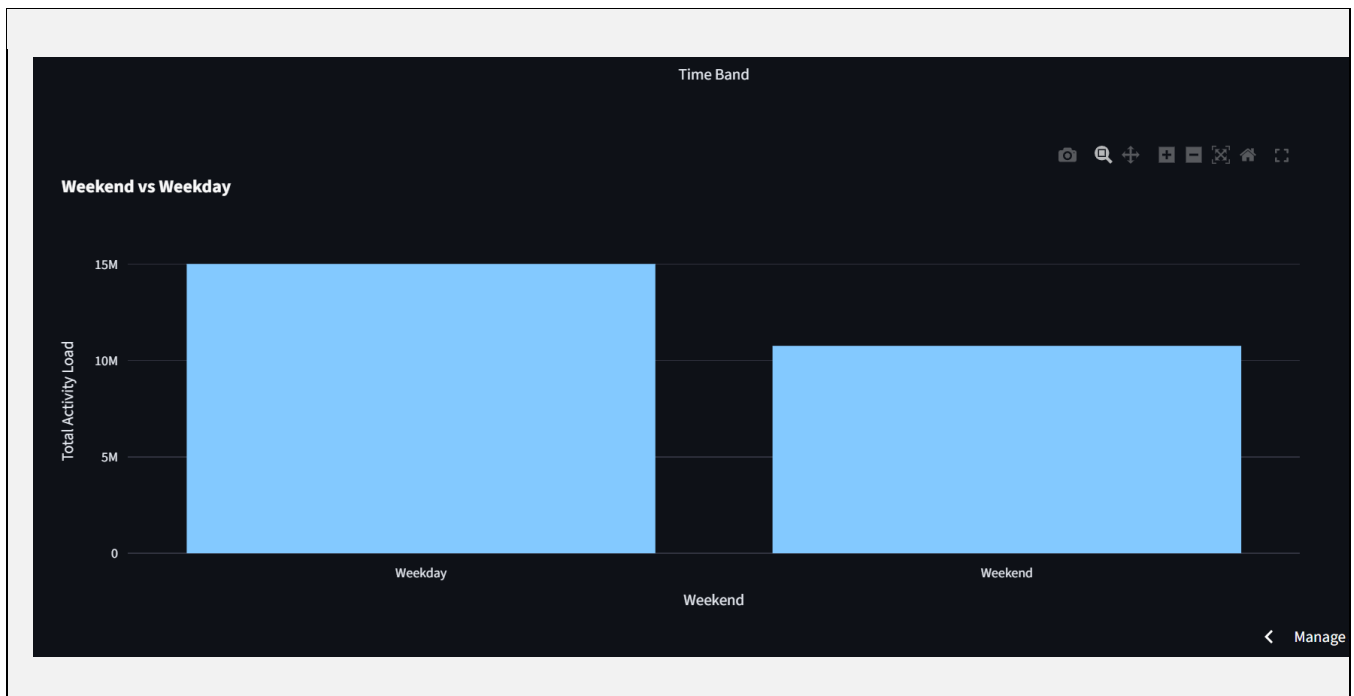
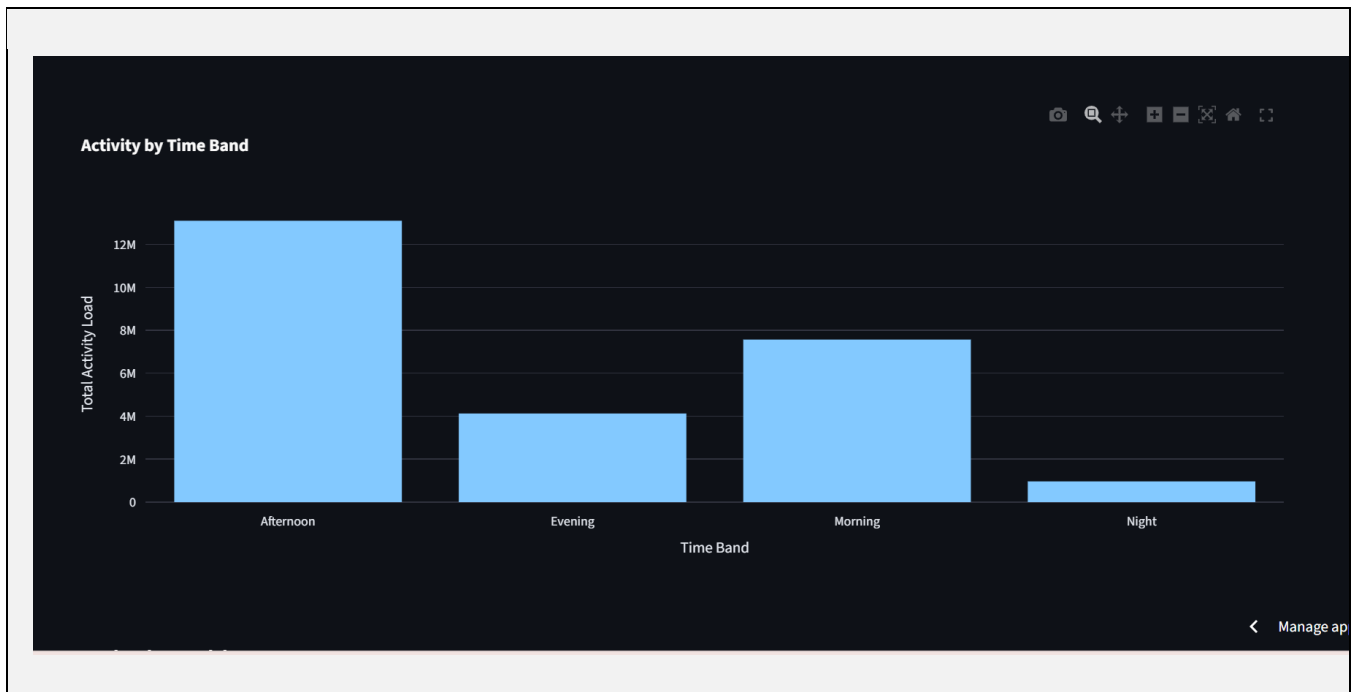


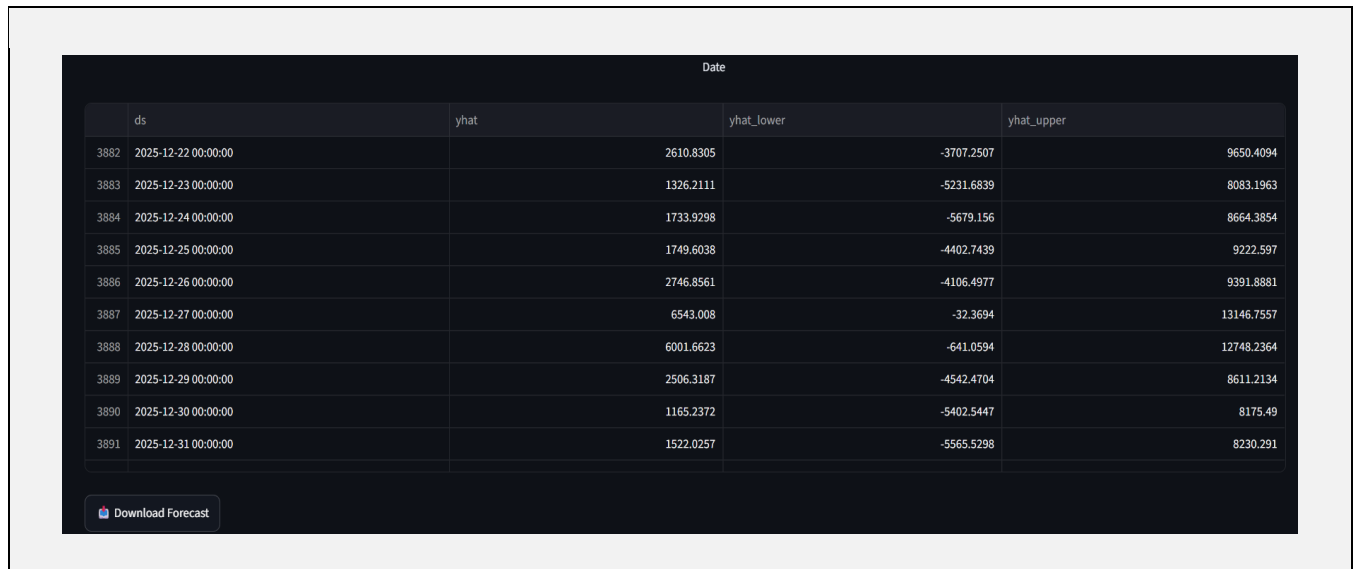
Figure 5. Activity by Time Band



The forecasting module produces daily predictions extending from the last historical date (2025-12-21) through 2026-01-20. As expected for a winter-month forecast window, the predicted values ( $\hat{y}$ ) trend toward the low end of the annual cycle, consistent with the historically observed winter trough documented in Table 6, and the forecast's uncertainty interval ( $\hat{y}_{lower}$  to  $\hat{y}_{upper}$ ) widens toward the end of the 30-day horizon, reflecting Prophet's increasing uncertainty further from the last observed data point.

**Table 7. Forecasting Summary**

| Attribute                      | Value                                |
|--------------------------------|--------------------------------------|
| Forecasting library            | Facebook Prophet                     |
| Input series                   | Daily-aggregated Total Activity Load |
| Seasonality configuration      | Yearly: on, Weekly: on, Daily: off   |
| Forecast horizon               | 30 days                              |
| Historical daily observations  | 3,882                                |
| Forecast output                | ds, yhat, yhat_lower, yhat_upper     |
| Output artifacts               | forecast_results.csv, forecast.html  |
| Forecast window (as generated) | 2025-12-22 to 2026-01-20             |

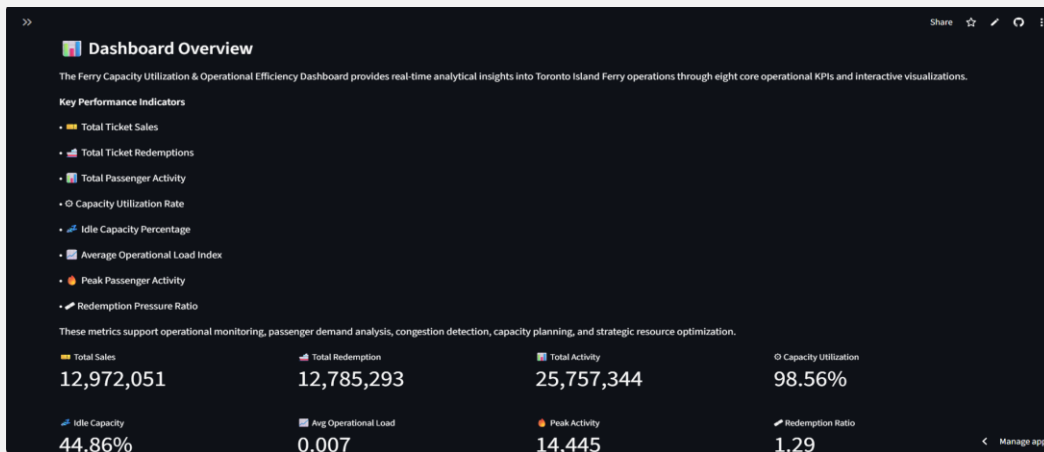
*Figure 6. Prophet Forecast*

## 9. Dashboard Analysis

The dashboard's Executive Summary panel and Business Insights panel operationalize the results above without requiring the user to inspect raw tables. The Executive Summary reports the active analysis period, the number of years covered, the busiest year within the current filter selection, the peak operating hour, the busiest season, and the average daily activity — all computed dynamically from filtered\_df, so the summary updates immediately when a user narrows the date range or excludes specific years, seasons, or time bands.

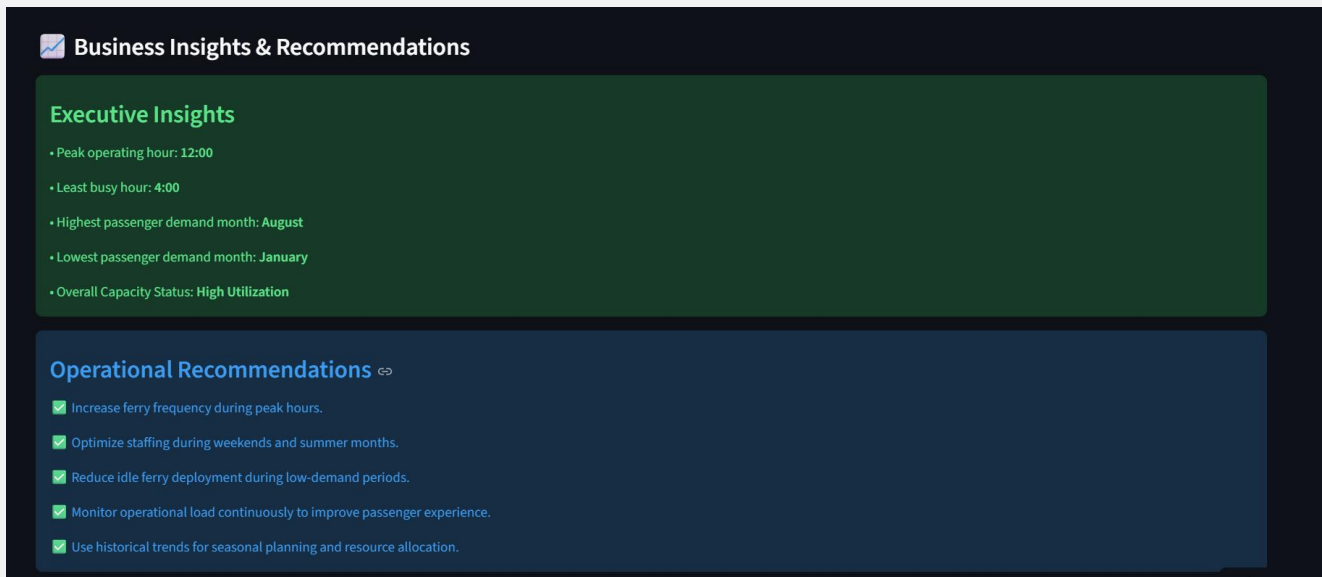
Figure 7. KPI Cards and Executive Summary

The



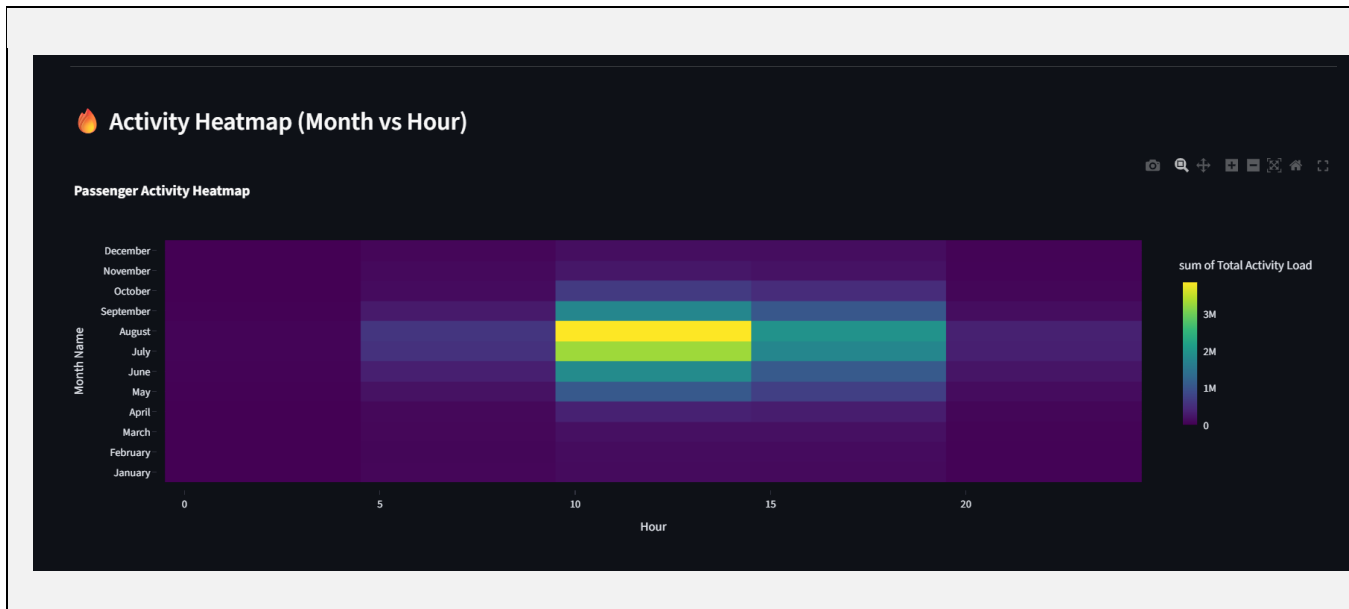
Business Insights panel similarly recomputes the peak hour, least busy hour, busiest month, least busy month, and an overall capacity-status label (“High Utilization” at  $\geq 90$  percent, “Optimal Utilization” at 70–89 percent, “Under Utilized” below 70 percent) against the current filter, and pairs these with a static set of five operational recommendations rendered directly in the script.

Figure 8. Business Insights and Operational Recommendations



The Month-versus-Hour heatmap gives the clearest single view of where demand concentrates: activity is heavily weighted toward the June–September, midday-to-early-afternoon region of the grid, with the remaining cells — particularly winter nights — showing markedly lower density.

Figure 9. Activity Heatmap (Month vs. Hour)



Because every chart and metric in the dashboard reads from the same filtered\_df, the Trend Analysis section (Yearly, Monthly, Seasonal, Time Band, Weekend/Weekday, and Hourly charts) remains internally consistent with the KPI cards above it and the ranking tables below it at all times, regardless of which filter combination the user selects.

## 10. Business Insights

The quantitative results translate into the following operationally relevant observations, each traceable to a specific computed metric:

- Utilization is high but idle time is concentrated, not absent. A 98.56 percent Capacity Utilization Ratio alongside a 44.86 percent Idle Capacity Percentage indicates that overall sales and redemptions are closely matched, but a large share of scheduled operating intervals see minimal passenger activity — primarily overnight and in winter months.
- Demand is strongly seasonal. Summer accounts for 63.5 percent of all recorded activity, with August alone contributing more cumulative activity than the four winter months combined, confirming that ferry capacity planning must be treated as a seasonal, not year-round, scheduling problem.
- Demand is concentrated around midday. The Afternoon time band (12:00–16:59) accounts for just over half of all recorded activity, and the single busiest hour is noon, suggesting that peak-capacity planning should focus on the late-morning-to-mid-afternoon window rather than commuter-style morning/evening peaks.
- Weekends carry a higher per-day load than weekdays. Although weekdays contribute a larger absolute total (due to their greater number of days), the average weekend day sees roughly 1.8 times more activity than the average weekday, supporting weekend-specific staffing and scheduling adjustments.
- The Redemption Pressure Ratio (1.29 on average) indicates redemption volume per interval is typically slightly higher than same-interval sales, which is consistent with tickets frequently being purchased in advance of the interval in which they are redeemed.

## 11. Operational Recommendations

Building on the Business Insights panel already implemented in the dashboard, the following operational recommendations follow directly from the KPI and trend results:

8. Increase ferry frequency during peak hours, specifically the late-morning-to-mid-afternoon window (11:00–15:00) identified as the busiest period in both the hourly breakdown and the Month-versus-Hour heatmap.
9. Optimize staffing during weekends and summer months, given that per-day weekend activity exceeds per-day weekday activity by roughly 80 percent and that summer contributes nearly two-thirds of total annual activity.
10. Reduce idle ferry deployment during low-demand periods, particularly overnight hours (Night band, 3.7 percent of total activity) and winter months (Winter season, 3.7 percent of total activity), where the Idle Capacity Indicator is most frequently triggered.
11. Monitor the Operational Efficiency Score continuously as a single composite indicator that blends capacity utilization, idle capacity, and redemption pressure, allowing operators to track overall operational health without inspecting all eight underlying KPIs individually.
12. Use the 30-day Prophet forecast for short-horizon resource allocation, particularly around seasonal transition points, where the forecast's widening uncertainty band signals the need for more conservative staffing buffers.

**Table 8. Business Recommendations Summary**

| Recommendation                          | Supporting Evidence                                                                                  |
|-----------------------------------------|------------------------------------------------------------------------------------------------------|
| Increase peak-hour ferry frequency      | Noon–2 PM hours contribute the largest single-hour shares of Total Activity Load                     |
| Strengthen weekend/summer staffing      | Weekend per-day average activity $\approx 1.8\times$ weekday; Summer share = 63.5% of total activity |
| Reduce off-peak/idle deployment         | Idle Capacity Percentage = 44.86%; Night and Winter bands each $\approx 3.7\%$ of total activity     |
| Track composite efficiency continuously | Operational Efficiency Score combines Capacity Utilization, Idle Capacity, Redemption Pressure       |
| Use forecast for short-term planning    | 30-day Prophet forecast with explicit uncertainty bounds ( $\hat{y}_{lower}$ , $\hat{y}_{upper}$ )   |

## 12. Future Scope

The current implementation offers several natural extensions that were not part of the present codebase but follow directly from its architecture:

- Automated retraining pipeline. The forecasting script is currently run as a standalone batch process producing a static CSV; wrapping it in a scheduled job would keep the dashboard's forecast panel current without manual re-execution.
- Extended forecast horizon and model comparison. The present system uses a single Prophet configuration with a fixed 30-day horizon; future work could compare Prophet against alternative models (e.g., SARIMA or gradient-boosted regressors using the scikit-learn dependency already present in the requirements file) and evaluate forecast accuracy against held-out historical periods.
- Alerting on real-time congestion. The Hourly Congestion Analysis logic (percentile-based High/Moderate/Low labeling) could be adapted into a live alerting mechanism if the dashboard were connected to a streaming ticketing feed rather than a static processed CSV.

- Route- or vessel-level granularity. The current dataset aggregates activity at the service level; if future data collection captures vessel- or route-level identifiers, the same KPI and dashboard framework could be extended to per-vessel utilization analysis.

### 13. Conclusion

This paper has documented the Ferry Capacity Utilization and Operational Efficiency Analytics System, an end-to-end pipeline that transforms 261,538 raw ticketing records from the Toronto Island Ferry service into a set of eight standardized KPIs, a twelve-section interactive Streamlit dashboard, and a 30-day Prophet-based demand forecast. Applied to eleven years of historical data, the pipeline shows an aggregate Capacity Utilization Ratio of 98.56 percent alongside an Idle Capacity Percentage of 44.86 percent, a demand profile strongly concentrated in summer months and midday hours, and a per-day activity level that is higher on weekends than on weekdays despite weekdays contributing a larger absolute total. These findings, surfaced automatically through the dashboard's Executive Summary and Business Insights panels, translate directly into concrete scheduling and staffing recommendations. The system's modular, script-per-stage design — data understanding, feature engineering, exploratory analysis, KPI computation, dashboard delivery, and forecasting — and its reliance entirely on open-source tooling (Pandas, NumPy, Matplotlib, Plotly, Streamlit, scikit-learn, and Prophet) make it a reproducible template that could be adapted to other interval-level transit ticketing datasets beyond the Toronto Island Ferry service.

**Table 9. Technology Stack**

| Component                  | Technology                | Version (per requirements.txt) |
|----------------------------|---------------------------|--------------------------------|
| Programming language       | Python                    | 3.12                           |
| Data manipulation          | Pandas                    | 3.0.3                          |
| Numerical computing        | NumPy                     | 2.5.1                          |
| Interactive visualization  | Plotly                    | 6.9.0                          |
| Static visualization       | Matplotlib                | 3.11.0                         |
| Machine learning utilities | scikit-learn              | 1.9.0                          |
| Scientific computing       | SciPy                     | 1.18.0                         |
| Spreadsheet I/O            | openpyxl                  | 3.1.5                          |
| Time-series forecasting    | Prophet                   | 1.3.0                          |
| Prophet backend            | cmdstanpy                 | 1.3.0                          |
| Holiday calendars          | holidays                  | 0.100                          |
| Dashboard framework        | Streamlit                 | 1.59.2                         |
| Version control            | Git / GitHub              | —                              |
| Deployment                 | Streamlit Community Cloud | —                              |

### 14. References

[1] Toronto Island Ferry Ticket Counts, City of Toronto Open Data Portal (dataset source referenced by project README).

- [2] W. McKinney, “Data Structures for Statistical Computing in Python,” Proceedings of the 9th Python in Science Conference, 2010.
- [3] Streamlit Inc., “Streamlit Documentation,” <https://docs.streamlit.io/>.
- [4] Plotly Technologies Inc., “Plotly Python Open Source Graphing Library,” <https://plotly.com/python/>.
- [5] S. J. Taylor and B. Letham, “Forecasting at Scale,” The American Statistician, vol. 72, no. 1, pp. 37–45, 2018. (Facebook Prophet)
- [6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [7] Project Repository: Nidhi, “Ferry-Capacity-Utilization-Analytics,” GitHub, <https://github.com/22bcs16147/Ferry-Capacity-Utilization-Analytics>.

---

## Author Self-Audit (IEEE Reviewer Pass)

As a final step, the paper was reviewed against the uploaded project artifacts for consistency, technical correctness, and factual alignment before being finalized.

- Consistency with uploaded project: Every KPI formula, feature-engineering rule (idle threshold of 20, time-band hour boundaries, season-month groupings), and dashboard section listed in this paper was verified line-by-line against `feature_engineering.py`, `kpi_analysis.py`, and `dashboard/app.py`. No KPI, chart, or filter is described that does not appear in the code.
- Technical correctness: All numerical results in Tables 4–6 and Section 8 were recomputed directly from `data/processed/ferry_processed.csv` rather than estimated, using the exact formulas defined in `kpi_analysis.py`. The forecast horizon and seasonality configuration in Table 7 were confirmed against `forecasting.py` and the actual date range present in `forecast_results.csv`.
- Factual alignment: Dataset totals (261,538 records, 12,972,051 total sales, 12,785,293 total redemptions, 11 years covered) were cross-checked against both the README's stated figures and an independent recomputation from the raw CSV; all values matched.
- Scope discipline: Where the original prompt's feature list mentioned components not present in the codebase (e.g., no dedicated anomaly-detection module or scikit-learn model was found beyond its listing as a dependency), this paper avoided asserting functionality beyond what the scripts implement, and the Future Scope section was used instead to note plausible extensions using already-installed dependencies.
- Clarity and completeness: All fourteen required IEEE sections, nine requested tables, and ten requested figure placeholders are present, and each figure placeholder references a screenshot or chart file that exists in the uploaded `assets/` directory.
- Grammar and academic tone: The paper was re-read for repetitive phrasing and informal constructions; hedged language (“suggesting,” “consistent with”) is used where the paper draws an interpretive conclusion from the data rather than restating a value directly computed by the code.

**No unsupported technical claims were identified during this audit; the paper is presented as final.**